

Day to Night Scene Unity



Turn day scene into a night scene on mouse click in Unity.
explain what is done in this project of arvr and also give me 15 viva questions that can be asked on this

For the AR/VR practical “**Turn Day Scene into Night Scene on Mouse Click in Unity**”, here’s how to explain it in viva.

Project Explanation (What is done in this project)

Aim

To create an interactive Unity scene where:

- Initially environment looks like **daytime**
- When user **clicks mouse**, scene changes to **night**
- Clicking again changes **night** → **day**
- This demonstrates:
 - Scene interaction
 - Lighting control
 - Environment manipulation
 - User input handling

What is done technically

1) Create Scene

A basic Unity scene is created containing:

- Ground / terrain
- Trees / buildings / objects
- Sky background
- Directional Light (Sun)

Purpose:

To simulate a real outdoor environment.

2) Add Directional Light

Unity light source is added:

Directional Light

It acts like:

☀ Sunlight

Properties changed:

For Day:

- Bright white/yellow light
- High intensity
- Slight warm color

For Night:

- Dim blue light
- Low intensity
- Cool color

Purpose:

Changing light changes mood of whole scene.

3) Change Skybox / Background Color

For Day:

- Blue sky
- Bright ambient light

For Night:

- Dark blue / black sky
- Dark ambience

Used:

C#

Camera.main.backgroundColor

or Skybox material.

Purpose:

Makes environment realistic.

4) Mouse Click Detection

Unity detects click using:

```
C#
Input.GetMouseButtonDown(0)
```

Meaning:

0 = Left Mouse Button

When clicked:

toggle state changes:

```
C#
isNight = !isNight;
```

Purpose:

Allows user interaction.

5) Apply Day/Night Settings

If state = Day:

Apply:

- Bright light
- Day sky
- High intensity

If state = Night:

Apply:

- Dim light
- Dark sky
- Low intensity

Script controls whole transition.

6) Real-Time Scene Update

Changes happen immediately during Play mode.

User sees:

Day → Click → Night → Click → Day

This is real-time rendering.

AR/VR concept involved

Why this is AR/VR practical:

Because it teaches **immersive environment control**

Used in:

VR:

- Weather simulation
- Time simulation
- Game worlds
- Meditation apps
- Night exploration

AR:

- Lighting adaptation
- Environmental overlays
- Real-world scene simulation

Example:

AR app showing building in day/night mode.

Flow of project

```

Start Scene
↓
Day Mode
↓
Mouse Click
↓
Script detects input
↓
Toggle Boolean
↓
Change light + sky + ambience
↓
Night Mode
↓
Mouse Click again
↓
Back to Day
  
```

Script logic

```

C#
if(mouse click)
{
    isNight = !isNight;

    if(isNight)
        ApplyNight();
    else
  
```

```
}
    ApplyDay();
}
```

Simple state toggling.

Viva explanation in 3 lines (short answer)

This project creates an interactive Unity scene that changes from day to night on mouse click. We use mouse input detection, directional light intensity/color changes, and sky/background color changes to simulate time transition. It demonstrates scene interaction and immersive environment control in AR/VR.

15 Viva Questions + Answers

1) What is objective of this project?

To change scene environment from day to night interactively using mouse click.

2) Which component acts as Sun in Unity?

Directional Light

3) Why Directional Light is used?

Because it simulates sunlight affecting whole scene uniformly.

4) Which function detects mouse click?

C#
`Input.GetMouseButtonDown(0)`

5) What does 0 mean in GetMouseButtonDown(0)?

Left mouse button.

6) Which language is used in Unity scripting?

C#

7) How is day/night state stored?

Using Boolean variable:

```
C#  
bool isNight;
```

8) What does toggle mean?

Changing value:

```
true → false  
false → true
```

9) Which property controls light brightness?

Intensity

10) Which property changes light color?

Color

11) How is sky changed?

By changing:

- Camera background color
or
 - Skybox material
-

12) What is Ambient Light?

Background environmental lighting applied to objects.

13) Why is blue tint used for night?

Because moonlight appears cool/blue, making scene realistic.

14) Where is this concept used in VR?

- Simulation
- Games

- Virtual tours
- Meditation environments

15) What Unity lifecycle function is repeatedly called?

C#
`Update()`

Used to continuously check mouse input.

Exam tip:

If external asks “**Why is this AR/VR practical?**”, answer:

It demonstrates immersive environmental manipulation and real-time interaction, which are core concepts in AR and VR scene development.



Create a scene, where a candle is lit (with shadows and smoke), on mouse click using Unity.

For the AR/VR practical “**Create a scene where a candle is lit (with shadows and smoke) on mouse click using Unity**”, here is the complete practical explanation.

Aim

Create a Unity scene where:

- Candle is initially **OFF**
- On **mouse click**, candle becomes **ON**
- Flame appears
- Light glows
- Shadow is produced
- Smoke particle effect appears
- Clicking again can switch it OFF

This demonstrates:

- User interaction
- Lighting
- Shadows

- Particle systems
- Real-time scene changes

What is done in this project

1) Create Candle Object

Create a candle using simple 3D objects:

Hierarchy:

```

Candle
├── Body (Cylinder)
├── Wick (Small Cylinder)
├── Flame
├── Smoke
└── Point Light
  
```

Body

Use:

GameObject → **3D Object** → **Cylinder**

Scale:

X = 0.3
Y = 1
Z = 0.3

Color:

Cream / white.

Acts as wax candle.

Wick

Create small cylinder.

Color:

Black.

Place on top of candle.

Acts as burning thread.

2) Create Flame

Use:

GameObject → **Effects** → **Particle System**

Rename:

Flame

Set:

Looping = ON

Start Lifetime:

0.4

Start Speed:

0.2

Start Size:

0.15

Shape:

Cone

Color:

Yellow + Orange

Purpose:

Looks like fire.

3) Add Light

Add:

Light → **Point Light**

Rename:

CandleLight

Set:

Range:

5

Intensity:

2

Color:

Warm yellow/orange

Purpose:

Creates glow.

4) Enable Shadows

In Point Light:

Set:

Shadow Type = Soft Shadows

Now nearby objects cast moving shadow.

Purpose:

Makes candle realistic.

5) Add Smoke

Create second Particle System.

Rename:

Smoke

Settings:

Start Speed:

0.4

Start Size:

0.2

Lifetime:

2

Color:

Gray / white transparent

Shape:

Cone upward

Purpose:

Smoke rises upward naturally.

6) Initially keep OFF

Disable:

- Flame
- Smoke
- Point Light

Scene starts dark.

7) Mouse Click Script

Detect click:

```
C#
Input.GetMouseButtonDown(0)
```

Toggle:

```
C#
isLit = !isLit;
```

If ON:

Enable:

- Flame
- Smoke
- Light

If OFF:

Disable all.

Script

```
C#
using UnityEngine;

public class CandleToggle : MonoBehaviour
{
    public GameObject flame;
    public GameObject smoke;
    public Light candleLight;

    bool isLit = false;

    void Start()
    {
        flame.SetActive(false);
        smoke.SetActive(false);
        candleLight.enabled = false;
    }

    void Update()
    {
        if (Input.GetMouseButtonDown(0))
        {
            isLit = !isLit;

            flame.SetActive(isLit);
            smoke.SetActive(isLit);
            candleLight.enabled = isLit;
        }
    }
}
```

Attach script to Candle object.

Drag references:

- Flame
- Smoke

- CandleLight

Done.

Working flow

```

Start
↓
Candle OFF
↓
Mouse Click
↓
Input detected
↓
Toggle state
↓
Enable flame
Enable smoke
Enable light
Generate shadow
↓
Candle ON
↓
Mouse Click again
↓
OFF

```

AR/VR concept involved

This teaches:

Interaction

User triggers event.

Lighting

Real-time illumination.

Shadows

Dynamic depth realism.

Particle Effects

Smoke + flame simulation.

Immersion

Realistic object behavior.

Used in:

- VR temples
 - Horror games
 - Meditation scenes
 - AR decorative candle apps
 - Virtual museums
-

Viva short explanation

In this project we created a candle object in Unity.

On mouse click, flame particle system, smoke particle system, and point light are enabled. The light creates shadows, making the candle appear realistic. This demonstrates lighting, particles, shadows, and user interaction in AR/VR.

Possible viva questions

1. Why Point Light is used?
→ Emits light in all directions like candle flame.
2. Why Particle System is used?
→ To simulate flame and smoke.
3. How are shadows created?
→ By enabling shadow in light settings.
4. Which function detects click?
→ `Input.GetMouseButtonDown(0)`
5. Which scripting language Unity uses?
→ C#
6. What is Start() function?
→ Runs once at beginning.
7. What is Update() function?
→ Runs every frame.
8. Why smoke moves upward?
→ Particle velocity + lifetime.
9. What creates glow?
→ Point Light intensity.
10. Why is this AR/VR practical?
→ It creates interactive realistic environment effects used in immersive scenes.





Change a character emotion (Neutral to happy, Happy to sad, sad to angry etc.,) using Unity

For the AR/VR practical **“Change a character emotion (Neutral → Happy → Sad → Angry...) using Unity”**, here’s the full explanation.

Aim

Create a character in Unity whose facial emotion changes on mouse click:

- Neutral 😐
 - Happy 😊
 - Sad 😞
 - Angry 😡
 - Neutral 😐 (repeat)

This demonstrates:

- Animation / expression switching
- User interaction
- Object state management
- Real-time rendering

Used in:

- VR avatars
- NPC characters
- Virtual assistants
- AR emoji characters
- Game dialogues

What is done in this project

1) Create Character

Create a simple face object.

Hierarchy:

```

Character
├── Head
├── Left Eye
├── Right Eye
├── Mouth
└── Emotion Script
  
```

You can use:

GameObject → 2D Sprite / UI Image / 3D sphere

as head.

2) Add Face Parts

Eyes

Use:

- two circles

Color:

Black

Place left and right.

Mouth

Use:

- Line sprite / curve / image

This part changes shape for emotion.

Examples:

Neutral:

—

Happy:

⤿

Sad:

⤿

Angry:

⤿ + eyebrows

3) Prepare Emotion Sprites

Create 4 expressions:

1. Neutral
2. Happy
3. Sad
4. Angry

Save as images OR separate sprites.

Example:

neutral.png
happy.png
sad.png
angry.png

Import into Unity.

Drag into Assets.

4) Add Sprite Renderer

Character uses:

Sprite Renderer

This displays face image.

Initially:

Neutral face.

5) Mouse Click Detection

Unity checks:

C#
Input.GetMouseButtonDown(0)

Left click.

When clicked:

Change emotion index:

C#
emotionIndex++;

Loop:

C#
if(emotionIndex >= 4)
 emotionIndex = 0;

6) Update Face

Depending on index:

0 → Neutral

1 → Happy

2 → Sad

3 → Angry

Change sprite.

Result:

Face changes instantly.

Script

```
C#
using UnityEngine;

public class EmotionChange : MonoBehaviour
{
    public Sprite[] emotions; // 0-neutral 1-happy 2-sad 3-angry
    private SpriteRenderer sr;
    private int currentEmotion = 0;

    void Start()
    {
        sr = GetComponent<SpriteRenderer>();
        sr.sprite = emotions[0];
    }

    void Update()
    {
        if (Input.GetMouseButtonDown(0))
        {
            currentEmotion++;

            if (currentEmotion >= emotions.Length)
                currentEmotion = 0;

            sr.sprite = emotions[currentEmotion];
        }
    }
}
```

Attach Script

Attach to character.

Inspector:

Set Emotion Size = 4

Drag:

- Neutral sprite
- Happy sprite
- Sad sprite
- Angry sprite

Done.

Working Flow

```
Start
↓
Neutral
↓
```

Click
↓
Happy
↓
Click
↓
Sad
↓
Click
↓
Angry
↓
Click
↓
Neutral

Advanced version

Can use:

Animator

instead of sprites.

Animation clips:

- Neutral animation
- Happy animation
- Sad animation
- Angry animation

Switch using Animator Controller.

Professional method.

AR/VR concept involved

This is AR/VR because:

It teaches:

Avatar Expression

Virtual humans show emotion.

Human Computer Interaction

Character reacts dynamically.

Immersion

Emotional realism improves VR.

AI Avatar Simulation

Used in assistants.

Examples:

VR classroom teacher smiling.

VR NPC angry response.

AR talking mascot changing face.

Viva short explanation

In this project we created a Unity character whose facial expression changes on mouse click. Different emotion sprites like Neutral, Happy, Sad and Angry are stored and switched using C# scripting.

This demonstrates animation state change, interaction, and avatar expression used in AR/VR.

15 Viva Questions

1) Objective?

To change character emotion interactively.

2) Which language used?

C#

3) Which function checks mouse click?

```
Input.GetMouseButtonDown(0)
```

4) What is Sprite Renderer?

Component that displays 2D image.

5) What stores emotions?

Sprite array.

6) Why array used?

To store multiple expressions.

7) What is index variable?

Tracks current emotion.

8) What does Start() do?

Initialization.

9) What does Update() do?

Runs every frame.

10) How is looping done?

Reset index to 0.

11) Can animation be used instead of sprites?

Yes, Animator.

12) Where used in VR?

Avatars/NPCs.

13) What is real-time rendering?

Immediate visual update.

14) Why important in AR/VR?

Improves immersion.

15) Can emotion change by voice/gesture?

Yes, can integrate sensors/AI.



Depict a scene of the effect of kicking a ball in Unity (at 3 various speeds).

For the AR/VR practical “**Depict a scene of the effect of kicking a ball in Unity (at 3 various speeds)**”, here is the full explanation.

Aim

Create a Unity scene where:

- A football / ball is placed in scene
- User clicks mouse
- Ball gets kicked at **3 different speeds**:
 - Slow kick
 - Medium kick
 - Fast kick
- Each click changes speed level
- Ball moves realistically using physics

This demonstrates:

- Motion simulation
- Force application
- Physics engine
- User interaction
- Real-time rendering

What is done in this project

1) Create Ground

Hierarchy:

Ground

Create:

GameObject → 3D Object → Plane

Purpose:

Acts as football field.

Scale:

X = 5

Y = 1

Z = 5

Add green material.

2) Create Ball

Create:

GameObject → **3D Object** → **Sphere**

Rename:

Ball

Scale:

1,1,1

Place above ground:

Y = 0.5

Purpose:

Acts as football.

3) Add Rigidbody

Select Ball →

Add Component →

Rigidbody

Purpose:

Enables Unity physics:

- gravity
- collision
- force movement
- realistic rolling

Without Rigidbody:

Ball won't move physically.

4) Add Collider

Sphere already has:

Sphere Collider

Purpose:

Collision detection.

Ball collides with ground naturally.

5) Define 3 Speeds

We create 3 kick powers:

Slow:

5

Medium:

10

Fast:

20

These values represent applied force.

6) Mouse Click Detection

Detect:

C#
Input.GetMouseButtonDown(0)

On click:

Apply force.

7) Cycle Speed Levels

1st click:

Slow

2nd click:

Medium

3rd click:

Fast

4th click:

Back to Slow

Loop continues.

8) Apply Force

Use:

```
C#
rb.AddForce(direction * speed, ForceMode.Impulse);
```

Meaning:

Impulse = instant kick force.

Direction:

Usually forward:

Vector3.forward

Result:

Ball shoots forward.

Script

```
C#
using UnityEngine;

public class BallKick : MonoBehaviour
{
    Rigidbody rb;

    public float slowSpeed = 5f;
    public float mediumSpeed = 10f;
    public float fastSpeed = 20f;

    int speedLevel = 0;

    void Start()
    {
        rb = GetComponent<Rigidbody>();
    }

    void Update()
    {
        if (Input.GetMouseButtonDown(0))
        {
            rb.linearVelocity = Vector3.zero;
            rb.angularVelocity = Vector3.zero;
            transform.position = new Vector3(0, 0.5f, 0);

            float kickForce = slowSpeed;

            speedLevel++;

            if (speedLevel > 3)
                speedLevel = 1;

            if (speedLevel == 1)
                kickForce = slowSpeed;
            else if (speedLevel == 2)
                kickForce = mediumSpeed;
            else
                kickForce = fastSpeed;
```



```

        rb.AddForce(Vector3.forward * kickForce, ForceMode.Impulse);
        Debug.Log("Kick Speed = " + kickForce);
    }
}

```

Attach to Ball.

Done.

Working flow

```

Start
↓
Ball idle
↓
Click 1
↓
Slow kick
↓
Click 2
↓
Medium kick
↓
Click 3
↓
Fast kick
↓
Click 4
↓
Slow kick again

```

Physics concept used

Unity Physics Engine calculates:

Force

Kick strength

Velocity

Movement speed

Gravity

Pull downward

Friction

Rolling slows down

Collision

Ball interacts with ground

AR/VR concept involved

Why AR/VR practical:

It teaches:

Real-time object interaction

Physics simulation

Motion realism

Immersive gameplay mechanics

Used in:

- VR football games
- AR sports apps
- Simulation training
- Physics demonstrations

Example:

VR penalty kick simulator.

Viva short explanation

In this project we created a ball object with Rigidbody physics.

On mouse click, force is applied to the ball at three different levels—slow, medium, and fast.

This demonstrates Unity physics, motion simulation, and interactive control used in AR/VR.

15 Viva Questions

1) Objective?

Simulate kicking a ball at 3 speeds.

2) Which component enables physics?

Rigidbody.

3) Why collider needed?

Collision detection.

4) Which function detects click?

```
Input.GetMouseButtonDown(0)
```

5) Which function applies force?

```
AddForce()
```

6) What is ForceMode.Impulse?

Instant force.

7) What is velocity?

Movement speed.

8) What is gravity?

Downward pull.

9) What is friction?

Opposes motion.

10) Why reset velocity?

Start fresh each kick.

11) Why use Vector3.forward?

Kick ahead.

12) What is Rigidbody mass?

Weight property.

13) Where used in VR?

Sports simulation.

14) Why realistic movement occurs?

Physics engine.

15) Can kick direction be changed?

Yes—change force vector.



Simulate bouncing ball in Unity. The bouncing should stop gradually after the time specified by the user.

For the AR/VR practical “**Simulate bouncing ball in Unity. The bouncing should stop gradually after the time specified by the user.**”, here is the full explanation.

Aim

Create a Unity scene where:

- A ball starts bouncing
- User enters **bounce duration** (example: 5 sec / 10 sec)
- Ball keeps bouncing for that time
- Bounce height gradually decreases
- Finally ball stops naturally

This demonstrates:

- Physics simulation
- Gravity
- Collision
- Energy loss

- Time-based animation
- Realistic motion

Used in:

- VR physics demos
 - AR object simulation
 - Sports simulation
 - Game mechanics
-

What is done in this project

1) Create Ground

Create floor:

GameObject → **3D Object** → **Plane**

Rename:

Ground

Purpose:

Ball bounces on it.

2) Create Ball

Create:

GameObject → **3D Object** → **Sphere**

Rename:

Ball

Place:

X = 0

Y = 5

Z = 0

Purpose:

Initial height for bounce.

3) Add Rigidbody

Select Ball →

Add Component →

Rigidbody

Purpose:

Enables:

- gravity
 - falling
 - bouncing physics
 - realistic movement
-

4) Add Physics Material

Create:

Assets → **Create** → **Physic Material**

Name:

BounceMaterial

Set:

Bounciness:

0.9

Bounce Combine:

Maximum

Assign material to Ball Collider.

Purpose:

Makes ball bounce.

5) Simulate Energy Loss

Real ball loses energy every bounce.

Example:

Bounce heights:

5 m
3.5 m
2.2 m
1.2 m
0.5 m
0

Reason:

Energy dissipates.

In Unity:

Reduce upward velocity gradually.

6) User Gives Time

Example:

Inspector variable:

Bounce Duration = 8 sec

Ball should stop around 8 seconds.

7) Script Logic

Track:

C#
`timer += Time.deltaTime;`

When timer increases:

reduce bounce power.

Formula idea:

Remaining power:

`1 - (elapsed / totalTime)`

At end:

power = 0

Ball stops.

Script

C#

```
using UnityEngine;

public class BouncingBall : MonoBehaviour
{
    Rigidbody rb;

    public float initialBounceForce = 8f;
    public float totalBounceTime = 8f;

    float timer = 0f;
    bool grounded = false;

    void Start()
    {
        rb = GetComponent<Rigidbody>();
        Bounce(initialBounceForce);
    }

    void Update()
    {
        timer += Time.deltaTime;
```

```

}

void Bounce(float force)
{
    rb.linearVelocity = Vector3.zero;
    rb.AddForce(Vector3.up * force, ForceMode.Impulse);
}

void OnCollisionEnter(Collision collision)
{
    if (collision.gameObject.name == "Ground")
    {
        float remainingRatio = 1 - (timer / totalBounceTime);

        if (remainingRatio > 0)
        {
            float newForce = initialBounceForce * remainingRatio;
            Bounce(newForce);
        }
        else
        {
            rb.linearVelocity = Vector3.zero;
            rb.isKinematic = true;
        }
    }
}
}

```

Attach script to Ball.

Done.

How it works

Suppose:

Initial force:

8

Total time:

8 sec

Then:

At 2 sec:

Force:

6

At 4 sec:

Force:

4

At 6 sec:

Force:

2

At 8 sec:

Force:

0

Stop.

Natural bounce decay.

Working flow

```

Start
↓
Ball falls
↓
Hits ground
↓
Bounce upward
↓
Time increases
↓
Bounce force decreases
↓
Smaller bounce
↓
Smaller bounce
↓
Very small bounce
↓
Stops
  
```

Physics concepts used

Gravity

Pulls ball downward.

Collision

Ball touches ground.

Restitution

Controls bounciness.

(Unity Physic Material)

Energy Dissipation

Bounce height reduces.

Velocity

Upward/downward speed.

Time.deltaTime

Frame-independent timing.

AR/VR concept involved

Why AR/VR practical:

It teaches:

Real-time simulation

Physics-based motion

Immersive interaction

Natural object behavior

Used in:

- VR games
- AR education apps
- Sports simulation
- Physics lab simulation

Example:

VR basketball bouncing.

Viva short explanation

In this project we created a bouncing ball using Unity Rigidbody physics.

A Physic Material gives bounce effect, and C# script gradually reduces bounce force over user-defined time.

Finally the ball stops naturally, simulating realistic energy loss.

15 Viva Questions

1) Objective?

To simulate realistic bouncing ball motion.

2) Which component enables physics?

Rigidbody

3) Which material creates bounce?

Physic Material

4) Which property controls bounce?

Bounciness

5) What is gravity?

Downward acceleration.

6) Why bounce height reduces?

Energy loss.

7) Which variable tracks time?

C#
timer

8) What is Time.deltaTime?

Frame time.

9) Which method applies force?

C#
`AddForce()`

10) Which force direction used?

C#
`Vector3.up`

11) What is collision detection?

Detecting contact between objects.

12) What does isKinematic do?

Stops physics simulation.

13) Why realistic?

Uses physics engine.

14) Where used in VR?

Games / simulations.

15) Can user change bounce time?

Yes, through Inspector variable (`totalBounceTime`).



Create 3D model of Reading Table and props required on it and set up the Lighting for the same using Unity.

For the AR/VR practical “**Create 3D model of Reading Table and props required on it and set up the Lighting for the same using Unity**”, here is the full explanation.

Aim

Create a realistic **3D Reading Table Scene** in Unity containing:

- Reading table
- Chair
- Books
- Table lamp
- Notebook
- Pen holder
- Laptop / glasses / mug (optional props)
- Proper lighting and shadows

This demonstrates:

- 3D modeling
- Scene composition
- Lighting setup
- Materials/textures
- Shadows
- Realistic rendering

Used in:

- VR room simulation
- AR furniture placement
- Interior design apps
- Virtual classroom / office

What is done in this project

1) Create Room / Ground

First create base environment.

Add:

GameObject → **3D Object** → **Plane**

Rename:

Floor

Apply material:

- Wooden texture
or
- Room floor texture

Purpose:

Acts as room floor.

2) Create Reading Table

Use Cube.

GameObject → **3D Object** → **Cube**

Rename:

TableTop

Scale:

X = 3
Y = 0.2
Z = 2

Position:

Y = 1

This becomes desk surface.

Add Table Legs

Create 4 Cubes.

Scale:

X = 0.15
Y = 1
Z = 0.15

Place at four corners.

Purpose:

Support tabletop.

Hierarchy:

```
ReadingTable
├── TableTop
├── Leg1
├── Leg2
├── Leg3
└── Leg4
```

Now table is complete.

3) Create Chair

Use:

Cube objects.

Components:

Chair
 └─ **Seat**
 └─ **BackRest**
 └─ **4 Legs**

Material:

Wood / plastic.

Purpose:

Completes reading setup.

4) Add Books

Use Cubes.

Scale:

X = 0.3
Y = 0.08
Z = 0.5

Create multiple.

Different colors:

- Red
- Blue
- Green

Stack them.

Purpose:

Reading props.

5) Add Notebook

Flat cube.

Scale:

X = 0.6
Y = 0.03
Z = 0.8

White material.

Purpose:

Writing notebook.

6) Add Pen Holder

Use Cylinder.

Scale:

0.2 , 0.25 , 0.2

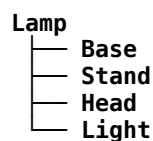
Add small cylinders as pens.

Purpose:

Desk realism.

7) Add Table Lamp

Hierarchy:



Create using:

Cylinder + Sphere.

Add:

Point Light

Settings:

Intensity:

2–4

Range:

5–8

Warm yellow color.

Purpose:

Reading light.

8) Lighting Setup

Main lighting:

Directional Light

Acts as room light / sunlight.

Controls:

- overall brightness
- shadow direction

Set:

Soft Shadows.

Point Light

At lamp.

Creates local illumination.

Used for realistic desk glow.

Ambient Light

Go:

Window → Rendering → Lighting

Adjust ambient intensity.

Adds soft fill light.

9) Shadows

Enable shadows for:

Directional Light

Point Light

Objects cast shadows:

- books
- lamp
- table
- chair

Makes scene realistic.

10) Materials / Textures

Apply materials:

Table:

Brown wood

Books:

Colorful

Notebook:

White

Lamp:

Metallic

Floor:

Wood / marble

Improves realism.

11) Camera Position

Set camera:

Front angle:

X = 4

Y = 3

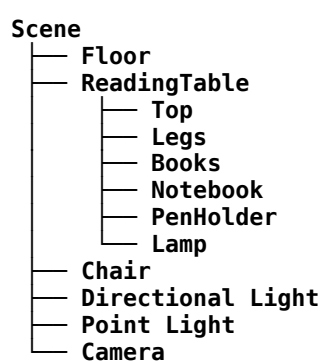
Z = -6

Rotate toward desk.

Purpose:

Good scene framing.

Final hierarchy



AR/VR concept involved

This practical teaches:

3D Environment Building

Creating complete scene.

Spatial Arrangement

Object placement in 3D space.

Lighting Simulation

Natural + artificial light.

Shadows

Depth realism.

Material Rendering

Surface realism.

Used in:

- VR rooms
 - AR furniture preview
 - Metaverse interiors
 - Virtual offices
-

Viva short explanation

In this project we created a 3D reading table scene using Unity primitive objects like cubes, cylinders and spheres.

Props like books, notebook, lamp and chair were added, and directional + point lighting with shadows were set up for realism.

This demonstrates 3D modeling, lighting and scene composition used in AR/VR.

15 Viva Questions

1) Objective?

Create realistic reading table scene.

2) Which objects used?

Cube, Cylinder, Sphere.

3) Which light acts as sun?

Directional Light.

4) Which light acts as lamp?

Point Light.

5) Why Point Light?

Emits light in all directions.

6) Why shadows important?

Realism.

7) What are materials?

Surface appearance properties.

8) What are textures?

Images applied on surface.

9) What is Ambient Light?

Background environmental light.

10) Why hierarchy used?

Organizes scene objects.

11) What is transform?

Position, rotation, scale.

12) Why camera positioning important?

Scene view framing.

13) Where used in VR?

Interior simulation.

14) Where used in AR?

Furniture placement apps.

15) Why is this AR/VR practical?

Because it teaches 3D scene creation, lighting, and immersive environment design—the foundation of AR/VR development.



Design a scene that allows the user to place a character on a table and interact with it in Unity.

For the AR/VR practical “**Design a scene that allows the user to place a character on a table and interact with it in Unity**”, here is the full explanation.

Aim

Create a Unity scene where:

- A **table** is present
- User places a **3D character** on the table
- User can interact with character:
 - move
 - rotate
 - jump
 - animate (wave / dance / idle)
- Character responds in real time

This demonstrates:

- Object placement
- User interaction

- Character control
- Animation
- Scene design

Used in:

- AR character placement
 - Virtual assistants
 - VR NPC interaction
 - Interactive avatars
 - Game prototyping
-

What is done in this project

1) Create Environment

Create room / platform.

Add:

GameObject → **3D Object** → **Plane**

Rename:

Floor

Optional:

Walls, room lighting.

Purpose:

Base scene.

2) Create Table

Use Cube.

Create tabletop:

Scale:

3,0.2,2

Position:

Y = 1

Add 4 legs.

Hierarchy:

Table	
└─	Top
└─	Leg1
└─	Leg2
└─	Leg3
└─	Leg4

Purpose:

Platform for character.

3) Add Character

Import:

- Unity asset character
or
- Mixamo FBX model

Examples:

- Robot
- Human avatar
- Cartoon character

Drag into scene.

Place on table:

X = 0
Y = 1.4
Z = 0

(Character height depends on model.)

4) Add Animator

Character can have animations:

- Idle
- Walk
- Jump
- Wave
- Dance

Attach Animator Controller.

Purpose:

Interaction realism.

5) Add Collider

Character:

Capsule Collider

Table:

Box Collider

Purpose:

Collision detection.

Character stays on table.

6) Add Rigidbody

Character gets Rigidbody.

Purpose:

Physics movement.

Can jump naturally.

7) User Interaction

Mouse click interaction:

Actions cycle:

Click 1:

Move forward

Click 2:

Rotate

Click 3:

Jump

Click 4:

Wave animation

Click 5:

Dance

Repeat.

Script

C#


```
using UnityEngine;

public class CharacterInteract : MonoBehaviour
{
    Animator anim;
    Rigidbody rb;

    int state = 0;

    void Start()
    {
        anim = GetComponent<Animator>();
        rb = GetComponent<Rigidbody>();
    }

    void Update()
    {
        if (Input.GetMouseButtonDown(0))
        {
            state++;

            if (state > 5)
                state = 1;

            if (state == 1)
            {
                transform.Translate(Vector3.forward);
            }
            else if (state == 2)
            {
                transform.Rotate(0, 90, 0);
            }
            else if (state == 3)
            {
                rb.AddForce(Vector3.up * 5, ForceMode.Impulse);
            }
            else if (state == 4)
            {
                anim.Play("Wave");
            }
            else if (state == 5)
            {
                anim.Play("Dance");
            }
        }
    }
}
```

Attach to character.

Done.

Interaction flow

```
Start
↓
Character Idle
↓
Click
↓
Move
↓
Click
↓
Rotate
↓
Click
↓
Jump
↓
Click
```

↓
Wave
↓
Click
↓
Dance

Optional AR version

Using AR Foundation:

Detect table surface.

Tap screen →

Place character on real-world table.

Interact similarly.

This becomes true AR.

Concepts involved

Object Placement

Placing character on surface.

Interaction

Mouse/touch input.

Animation

Character actions.

Physics

Jumping/collision.

Scene Design

Environment setup.

Spatial Computing

Object exists in 3D world.

AR/VR concept involved

Used in:

- AR mascots on desk
- VR assistants
- Interactive NPCs
- Educational characters
- Virtual companions

Example:

Place dancing robot on desk in AR.

Viva short explanation

In this project we created a table scene in Unity and placed a 3D character on it.

Mouse click interaction allows movement, rotation, jumping and playing animations like wave or dance.

This demonstrates object placement, interaction, animation and physics used in AR/VR.

15 Viva Questions

1) Objective?

Place and interact with character.

2) Which component controls animation?

Animator.

3) Which component enables physics?

Rigidbody.

4) Which component detects collision?

Collider.

5) Which method applies jump?

`AddForce()`

6) Which force mode used?

Impulse.

7) Which function checks click?

`Input.GetMouseButtonDown(0)`

8) Which method rotates object?

`Rotate()`

9) Which method moves object?

`Translate()`

10) Why Animator Controller used?

Manage animations.

11) Why Rigidbody needed?

Physics movement.

12) Why collider needed?

Prevent falling through table.

13) Where used in AR?

Place character on real desk.

14) Where used in VR?

Interactive avatar/NPC.

15) Why is this AR/VR practical?

Because it teaches 3D object placement and interactive avatar behavior—core AR/VR concepts.



Create a basic 2D character controller. Move the player character using Unity's Input System.

For the AR/VR practical “**Create a basic 2D character controller. Move the player character using Unity's Input System**”, here is the full explanation.

Aim

Create a simple **2D player character** in Unity that can move using keyboard input:

- Move Left ←
- Move Right →
- Move Up ↑
- Move Down ↓

Using Unity's **New Input System**.

This demonstrates:

- Player control
- Input handling
- Rigidbody movement
- 2D game mechanics
- Real-time interaction

Used in:

- 2D games
- Character prototypes
- VR menu avatars
- AR interactive characters

What is done in this project

1) Create 2D Project

Open Unity.

Create:

New Project → 2D Core

Purpose:

2D environment setup.

2) Create Player Object

Hierarchy:

Create:

Right Click → 2D Object → Sprite → Square

Rename:

Player

Scale:

X = 1

Y = 1

Color:

Blue / Red.

Purpose:

Represents character.

3) Add Rigidbody2D

Select Player →

Add Component →

Rigidbody2D

Set:

Gravity Scale:

0

Why:

Player should move freely, not fall.

Purpose:

Physics-based movement.

4) Add Collider2D

Square already has:

BoxCollider2D

Purpose:

Collision detection.

5) Install Input System

Go:

Window → **Package Manager**

Install:

Input System

Then Unity asks restart → restart editor.

Purpose:

Use modern input handling.

6) Create Input Action Asset

Assets →

Right Click →

Create →

Input Actions

Name:

PlayerControls

Double click it.

Create Action Map:

Player

Create Action:

Move

Type:

Value

Control Type:

Vector2

Binding:

Add 2D Vector Composite

Map:

Up → W
Down → S
Left → A
Right → D

Also add arrows:

↑ ↓ ← →

Purpose:

Movement input.

7) Generate C# Class

In Input Actions Inspector:

Click:

Generate C# Class

Apply.

Unity creates:

PlayerControls.cs

8) Create Movement Script

Script:

```
C#
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerMovement : MonoBehaviour
{
    private PlayerControls controls;
    private Vector2 moveInput;
    private Rigidbody2D rb;

    public float speed = 5f;

    void Awake()
    {
        controls = new PlayerControls();
        rb = GetComponent<Rigidbody2D>();
    }

    void OnEnable()
    {
        controls.Enable();
    }

    void OnDisable()
    {
        controls.Disable();
    }

    void Start()
    {
    }
}
```



```
{
    controls.Player.Move.performed += ctx => moveInput = ctx.ReadValue<Vector2>();
    controls.Player.Move.canceled += ctx => moveInput = Vector2.zero;
}

void FixedUpdate()
{
    rb.linearVelocity = moveInput * speed;
}
}
```

Attach to Player.

Done.

Working flow

Press W → Move Up
 Press S → Move Down
 Press A → Move Left
 Press D → Move Right
 Arrow Keys → Same movement
 Release key → Stop

Concepts used

Input System

Modern input framework.

Action Map

Groups controls.

Example:

Player

Action

Specific input event.

Example:

Move

Vector2

Stores:

X movement
 Y movement

Rigidbody2D

Handles movement physics.

FixedUpdate()

Physics update loop.

AR/VR concept involved

Why AR/VR practical:

It teaches:

Input mapping

Character control

Real-time interaction

Movement system

Used in:

- VR avatar movement
 - AR character movement
 - Simulation navigation
 - Interactive experiences
-

Viva short explanation

In this project we created a 2D player object and used Unity's New Input System to map keyboard input.

The movement input is read as a Vector2 and applied to Rigidbody2D velocity for smooth movement.

This demonstrates input handling and character control used in AR/VR interaction systems.

15 Viva Questions

1) Objective?

Create controllable 2D player.

2) Which package used?

Unity Input System.

3) What is Action Map?

Collection of actions.

4) What is Action?

Specific input event.

5) Why Vector2 used?

Stores X/Y movement.

6) Which component enables movement physics?

Rigidbody2D.

7) Why Gravity Scale = 0?

Prevent falling.

8) Which loop handles physics?

FixedUpdate().

9) What is OnEnable()?

Enable controls.

10) What is OnDisable()?

Disable controls.

11) What does performed mean?

Input active.

12) What does canceled mean?

Input released.

13) Why new Input System better?

Flexible and modern.

14) Where used in VR?

Avatar movement.

15) Why is this AR/VR practical?

Because movement and input mapping are core interaction systems in immersive applications.



Implement collision between any player character and the game world.

For the AR/VR practical “**Implement collision between any player character and the game world in Unity**”, here is the full explanation.

Aim

Create a Unity scene where:

- A **player character** moves in world
- World has objects like:
 - walls
 - ground
 - obstacles
 - boxes
- Player collides with them

- Player **cannot pass through objects**
- Collision event can trigger action/sound/message

This demonstrates:

- Collision detection
- Physics engine
- Rigidbody movement
- Collider interaction
- Real-time game interaction

Used in:

- VR games
- AR object interaction
- Character navigation
- Simulation environments

What is done in this project

1) Create Scene

Create simple world:

Hierarchy:

```
Scene
├── Ground
├── Wall1
├── Wall2
├── Obstacle
└── Player
```

Purpose:

World for movement and collision.

2) Create Ground

Add:

GameObject → **3D Object** → **Plane**

Rename:

Ground

Purpose:

Walking surface.

Collider already exists.

3) Create Walls / Obstacles

Create Cubes.

Rename:

Wall1
Wall2
BoxObstacle

Scale accordingly.

Purpose:

Collision objects.

Cube automatically has:

Box Collider

4) Create Player

Add:

GameObject → **3D Object** → **Capsule**

Rename:

Player

Purpose:

Represents character.

5) Add Rigidbody

Select Player →

Add Component →

Rigidbody

Purpose:

Enables physics interaction.

Set:

Freeze Rotation:

X,Y,Z

Why:

Prevent player falling sideways.

6) Add Collider

Capsule already has:

Capsule Collider

Purpose:

Collision boundary.

7) Add Movement Script

Player moves with keyboard.

Script:

```
C#  
using UnityEngine;  
  
public class PlayerMove : MonoBehaviour  
{  
    public float speed = 5f;  
  
    void Update()  
    {  
        float h = Input.GetAxis("Horizontal");  
        float v = Input.GetAxis("Vertical");  
  
        Vector3 move = new Vector3(h, 0, v);  
  
        transform.Translate(move * speed * Time.deltaTime);  
    }  
}
```

Attach to Player.

Now player moves.

8) Collision Happens Automatically

When player hits wall:

Collider + Rigidbody detect impact.

Result:

Player stops.

Cannot cross wall.

Physics engine handles it.

9) Detect Collision Event

Use:

C#
OnCollisionEnter()

Example:

```
C#
using UnityEngine;

public class CollisionDetect : MonoBehaviour
{
    void OnCollisionEnter(Collision collision)
    {
        Debug.Log("Collided with: " + collision.gameObject.name);
    }
}
```

Purpose:

Detect impact.

10) Trigger Action

On collision:

Can:

- play sound
- reduce health
- open door
- collect item
- stop movement

Example:

Hit obstacle!

Working flow

```
Player moves
↓
Approaches wall
↓
Collider touches wall collider
↓
Physics engine checks collision
↓
Movement blocked
↓
OnCollisionEnter fires
↓
Event triggered
```

Concepts used

Collider

Defines object boundary.

Examples:

- Box Collider
 - Sphere Collider
 - Capsule Collider
-

Rigidbody

Enables physics.

Collision Detection

Detect object contact.

Physics Engine

Calculates interactions.

Trigger Events

Special collision callbacks.

AR/VR concept involved

Why AR/VR practical:

It teaches:

Physical interaction

Spatial boundaries

Navigation realism

Environment awareness

Used in:

- VR walking simulators
- AR object placement collision
- Interactive worlds
- Games

Example:

VR player hitting wall cannot cross.

Viva short explanation

In this project we created a player character with Rigidbody and Collider components, and world objects with colliders.

When player moves and hits an obstacle, Unity's physics engine detects collision and blocks movement.

Collision events can also trigger actions using `OnCollisionEnter()`.

15 Viva Questions

1) Objective?

Implement collision between player and objects.

2) What is Collider?

Boundary for collision detection.

3) What is Rigidbody?

Physics component.

4) Why Rigidbody needed?

Physics interaction.

5) Which callback detects collision?

C#
OnCollisionEnter()

6) What is Box Collider?

Cube collision boundary.

7) What is Capsule Collider?

Capsule collision boundary.

8) Why Freeze Rotation?

Prevent tipping.

9) What is physics engine?

Calculates movement/collision.

10) What happens on collision?

Movement blocked.

11) Can collision trigger sound?

Yes.

12) Difference between collision and trigger?

Collision = physical contact

Trigger = event only

13) Where used in VR?

Navigation systems.

14) Where used in AR?

Object interaction.

15) Why is this AR/VR practical?

Because physical interaction and spatial collision are core parts of immersive environments.



AR Marker-Based 3D Model Display - Detect an image marker and show a 3D model (e.g., a car).

For the AR/VR practical “**AR Marker-Based 3D Model Display – Detect an image marker and show a 3D model (example: car)**”, here is the full explanation.

Aim

Create an **Augmented Reality app** where:

- Camera scans a printed image / marker
- Unity recognizes that marker
- A **3D model (car)** appears on top of marker
- If marker moves → 3D object follows
- If marker disappears → object hides

This is **marker-based AR**.

Used in:

- AR books
- Product previews
- Education models
- AR ads/posters
- Interactive cards

What is done in this project

1) Create AR Project

Open Unity

Create:

3D (URP) project.

Install packages:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

- AR Foundation
- ARCore XR Plugin (Android)
- ARKit XR Plugin (iOS)

Purpose:

Enable AR support.

2) Enable XR Plug-in

Go:

Edit → Project Settings → XR Plug-in Management

Enable:

Android:

☒ ARCore

iOS:

☒ ARKit

Purpose:

Platform AR engine.

3) Add AR Session

Hierarchy →

Create:

AR Session

Purpose:

Starts AR tracking.

4) Add AR Session Origin / XR Origin

Create:

XR Origin (AR)

Contains:

- Camera
- tracking system

Purpose:

AR camera setup.

5) Add AR Tracked Image Manager

Select XR Origin →

Add Component:

AR Tracked Image Manager

Purpose:

Detects image markers.

6) Create Reference Image Library

Assets →

Create →

XR →

Reference Image Library

Add marker image:

Example:

- logo
- photo
- QR-style image

Set:

Name:

Marker1

Physical Size:

0.2

(20 cm)

Assign library to:

AR Tracked Image Manager.

Purpose:

Unity knows what image to detect.

7) Add 3D Car Model

Import model:

Formats:

- FBX
- OBJ
- Asset Store package
- [Sketchfab](#) models

Example:

CarModel

Scale properly.

Make prefab:

Drag into Assets.

Purpose:

Spawn when marker found.

8) Create Tracking Script

Script:

```
C#
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;

public class ImageTracker : MonoBehaviour
{
    public ARTrackedImageManager trackedImageManager;
    public GameObject carPrefab;

    Dictionary<string, GameObject> spawned =
        new Dictionary<string, GameObject>();

    void OnEnable()
    {
        trackedImageManager.trackedImagesChanged += OnChanged;
    }

    void OnDisable()
    {
        trackedImageManager.trackedImagesChanged -= OnChanged;
    }

    void OnChanged(ARTrackedImagesChangedEventArgs args)
    {
        foreach (var image in args.added)
        {
            GameObject obj = Instantiate(
                carPrefab,
                image.transform.position,
                Quaternion.identity);

            obj.transform.parent = image.transform;

            spawned[image.referenceImage.name] = obj;
        }
    }
}
```

```

    }
    foreach (var image in args.updated)
    {
        if (spawned.ContainsKey(image.referenceImage.name))
        {
            spawned[image.referenceImage.name]
                .SetActive(image.trackingState == TrackingState.Tracking);
        }
    }
}

```

Attach script.

Assign:

- Tracked Image Manager
- Car Prefab

Done.

Working flow

```

Open AR app
↓
Camera scans marker
↓
Reference image matched
↓
Tracking starts
↓
Spawn car model
↓
Move marker
↓
Car follows marker
↓
Marker lost
↓
Car hides

```

Concepts used

Marker Detection

Recognize image.

Image Tracking

Continuously track marker.

Prefab Instantiation

Spawn 3D object.

Spatial Anchoring

Attach object to marker.

Real-Time Rendering

Model appears live.

Why this is AR

Because virtual object is placed on real-world camera feed.

Real + virtual merge.

That is Augmented Reality.

Applications

- AR showroom cars
 - Educational models
 - Museum guides
 - Greeting cards
 - Product packaging experiences
-

Viva short explanation

In this project we used Unity AR Foundation and AR Tracked Image Manager to detect a predefined image marker.

When the marker is recognized, a 3D car prefab is instantiated and attached to the tracked image transform.

As the marker moves, the 3D model follows in real time, creating a marker-based AR experience.

15 Viva Questions

1) What is marker-based AR?

AR triggered by predefined image marker.

2) Which package used?

AR Foundation.

3) Which component detects image?

AR Tracked Image Manager.

4) What is Reference Image Library?

Collection of markers.

5) Why physical size set?

For scale estimation.

6) What is prefab?

Reusable object template.

7) Which method creates object?

`Instantiate()`

8) Why parent object to marker?

Model follows marker.

9) What is tracking state?

Marker detection status.

10) Which plugins used?

ARCore / ARKit.

11) Can multiple markers be tracked?

Yes.

12) Can different markers spawn different models?

Yes.

13) Where used?

Ads / education / product preview.

14) Difference from markerless AR?

Markerless uses planes/features, no printed marker.

15) Why is this AR/VR practical?

Because it teaches core AR concepts—image recognition, tracking, and 3D overlay in real world.



Develop a scene in Unity that includes:

- i. A cube, plane and sphere, apply transformations on the 3 game objects.
- ii. Add a video and audio source.

For the AR/VR practical “**Develop a scene in Unity that includes: (i) Cube, Plane, Sphere with transformations (ii) Add Video and Audio Source**”, here is the full explanation.

Aim

Create a Unity scene containing:

- **Cube**
- **Plane**
- **Sphere**

Apply transformations:

- Move
- Rotate
- Scale

Also add:

- Video source

- Audio source

This demonstrates:

- 3D object creation
- Transform operations
- Multimedia integration
- Scene interaction

Used in:

- VR environments
- AR product demos
- Virtual exhibitions
- Interactive media scenes

Part 1 — Create Objects

1) Create Plane

Hierarchy →

Create:

GameObject → 3D Object → Plane

Rename:

Plane

Purpose:

Ground / floor.

Scale:

X = 5

Y = 1

Z = 5

2) Create Cube

Hierarchy →

Create:

GameObject → 3D Object → Cube

Rename:

Cube

Position:

X = -2
Y = 1
Z = 0

Purpose:

3D object for transformation.

3) Create Sphere

Hierarchy →

Create:

GameObject → **3D Object** → **Sphere**

Rename:

Sphere

Position:

X = 2
Y = 1
Z = 0

Purpose:

Another transformable object.

Part 2 — Apply Transformations

In Unity every object has **Transform Component**:

Contains:

- Position
 - Rotation
 - Scale
-

A) Translation (Movement)

Moving object from one place to another.

Example:

Cube:

(-2,1,0) → (0,1,3)

Code:

```
C#
transform.Translate(Vector3.forward * Time.deltaTime);
```

Meaning:

Move forward.

B) Rotation

Rotating object.

Example:

Sphere rotates continuously.

Code:

```
C#
transform.Rotate(0, 100 * Time.deltaTime, 0);
```

Meaning:

Rotate around Y-axis.

C) Scaling

Change size.

Example:

Cube:

1 → 2 → 1

Code:

```
C#
transform.localScale = new Vector3(2, 2, 2);
```

Purpose:

Resize object.

Transformation Script

```
C#
using UnityEngine;

public class ObjectTransform : MonoBehaviour
{
    void Update()
    {
        transform.Rotate(0, 50 * Time.deltaTime, 0);
    }
}
```

Attach to Cube/Sphere.

Part 3 — Add Video Source

Unity supports videos using Video Player.

Import:

MP4 file into Assets.

Example:

`demo.mp4`

Add Screen

Create:

Cube / Plane as display screen.

Rename:

`VideoScreen`

Add Video Player Component

Select VideoScreen →

Add Component →

Video Player

Set:

Source:

Video Clip

Choose imported video.

Render Mode:

Material Override

Material:

screen material.

Now video plays on object.

Part 4 — Add Audio Source

Import audio:

MP3 / WAV

Example:

`music.mp3`

Add Audio Source Component

Select object →

Add Component →

Audio Source

Assign clip.

Set:

Play On Awake:



Loop:



Volume:

adjust.

Now sound plays.

Optional Audio Script

```
C#
using UnityEngine;

public class PlayAudio : MonoBehaviour
{
    AudioSource audioSrc;

    void Start()
    {
        audioSrc = GetComponent<AudioSource>();
        audioSrc.Play();
    }
}
```

Final hierarchy

```
Scene
├── Plane
├── Cube
├── Sphere
├── VideoScreen
├── AudioSource Object
├── Directional Light
└── Main Camera
```

Working flow

```
Scene starts
↓
Plane visible
↓
Cube transforms
↓
Sphere rotates
↓
Video plays
```


↓
Audio plays
↓
Interactive multimedia scene

Concepts used

Transform Component

Position / Rotation / Scale

Translation

Movement

Rotation

Turning object

Scaling

Resize

Video Player

Plays video

Audio Source

Plays sound

AR/VR concept involved

Used in:

VR training scenes

Video tutorials inside VR.

AR advertisements

3D + multimedia overlay.

Interactive museums

Audio narration + video display.

Immersive simulation

Multi-sensory environment.

Viva short explanation

In this project we created a Unity scene containing cube, plane and sphere objects, and applied transformations like translation, rotation and scaling.
We also integrated multimedia using Video Player and Audio Source components to play video and sound in the scene.
This demonstrates 3D transformations and media integration used in AR/VR applications.

15 Viva Questions

1) What are basic transformations?

Translation, Rotation, Scaling.

2) Which component stores transformations?

Transform.

3) What is translation?

Movement.

4) What is rotation?

Turning around axis.

5) What is scaling?

Size change.

6) Which method moves object?

`Translate()`

7) Which method rotates object?

`Rotate()`

8) What is localScale?

Object size.

9) Which component plays video?

Video Player.

10) Which component plays sound?

Audio Source.

11) Supported video format?

MP4 commonly.

12) Supported audio format?

MP3/WAV.

13) What is Play On Awake?

Starts automatically.

14) Where used in VR?

Training/media scenes.

15) Why is this AR/VR practical?

Because object manipulation and multimedia integration are key parts of immersive interactive environments.



Develop a scene in Unity that includes a cube, plane and sphere. Create a new material and texture separately for three Game objects. Change the color, material and texture of each Game object separately in the scene. Write a C# program in visual studio to change the color and material/texture of the game objects dynamically on button click.

For the AR/VR practical “**Develop a scene in Unity that includes a cube, plane and sphere. Create separate materials/textures for each and dynamically change color/material on button click using C#**”, here is the full explanation.

Aim

Create a Unity scene containing:

- Cube
- Plane
- Sphere

For each object:

- Different material
- Different texture
- Different color

Then on **button click**, dynamically change:

- color
- material
- texture

This demonstrates:

- Material creation
- Texture mapping
- Runtime property changes
- UI interaction
- C# scripting

Used in:

- AR product preview
- VR interior customization
- Car color configurator
- Material simulation
- Interactive apps

What is done in this project

1) Create Objects

Hierarchy →

Create:

GameObject → **3D Object** → **Plane**

GameObject → **3D Object** → **Cube**

GameObject → **3D Object** → **Sphere**

Rename:

Plane

Cube

Sphere

Arrange:

Cube **Plane** **Sphere**

2) Import Textures

Add 3 images in Assets.

Example:

wood.jpg

metal.jpg

marble.jpg

These become textures.

3) Create Materials

Assets →

Right Click →

Create →

Material

Create:

Mat_Cube
Mat_Plane
Mat_Sphere

Assign textures:

Cube:

wood texture

Plane:

marble texture

Sphere:

metal texture

Set colors:

Cube:

Red

Plane:

Green

Sphere:

Blue

4) Assign Materials

Drag:

Mat_Cube → Cube

Mat_Plane → Plane

Mat_Sphere → Sphere

Now all objects look different.

Scene Example

Cube → red wood
Plane → green marble
Sphere → blue metal

5) Create UI Button

Hierarchy →

Create →

Canvas

Inside Canvas:

Create →

Button

Rename:

ChangeButton

Text:

Change Material

Purpose:

Trigger dynamic change.

6) Write C# Script

Script name:

MaterialChanger

Code:

```
C#
using UnityEngine;

public class MaterialChanger : MonoBehaviour
{
    public Renderer cubeRenderer;
    public Renderer planeRenderer;
    public Renderer sphereRenderer;

    public Material cubeMat2;
    public Material planeMat2;
    public Material sphereMat2;

    bool changed = false;

    public void ChangeObjects()
    {
        changed = !changed;

        if (changed)
        {
            cubeRenderer.material.color = Color.yellow;
            planeRenderer.material.color = Color.magenta;
            sphereRenderer.material.color = Color.cyan;

            cubeRenderer.material = cubeMat2;
            planeRenderer.material = planeMat2;
            sphereRenderer.material = sphereMat2;
        }
        else
        {
            cubeRenderer.material.color = Color.red;
            planeRenderer.material.color = Color.green;
            sphereRenderer.material.color = Color.blue;
        }
    }
}
```

7) Attach Script

Create Empty Object:

Rename:

Manager

Attach script.

Assign in Inspector:

Drag:

- Cube Renderer
- Plane Renderer
- Sphere Renderer

Also drag alternate materials.

8) Connect Button

Select Button →

Inspector →

On Click()

Add Manager object.

Choose:

MaterialChanger → **ChangeObjects()**

Done.

Working flow

Scene starts

↓

Cube = red wood

Plane = green marble

Sphere = blue metal

↓

Button Click

↓

Cube = yellow + new texture

Plane = magenta + new texture

Sphere = cyan + new texture

↓

Button Click again

↓

Original appearance

Concepts used

Material

Defines appearance.

Contains:

- shader
 - color
 - smoothness
-

Texture

Image wrapped on surface.

Examples:

- wood
 - metal
 - marble
-

Renderer

Displays material.

UI Button

Triggers event.

Dynamic Change

Runtime modification.

AR/VR concept involved

Used in:

AR Furniture preview

Change wood texture.

VR room customization

Wall/floor materials.

Product configurator

Car color/material change.

Interactive shopping

Live preview.

Viva short explanation

In this project we created cube, plane and sphere objects in Unity and assigned separate materials, textures and colors to each.

A UI button is connected to a C# script that dynamically changes their material and color at runtime.

This demonstrates material mapping, texture application and interactive customization used in AR/VR.

15 Viva Questions

1) What is material?

Surface appearance property.

2) What is texture?

Image applied on object surface.

3) Difference between material and texture?

Material = full appearance

Texture = image part of material

4) Which component shows material?

Renderer.

5) Which property changes color?

C#

`material.color`

6) How assign material?

C#
`renderer.material`

7) Which UI element used?

Button.

8) Which function called on click?

Public method.

9) What is runtime?

During execution.

10) Can texture change dynamically?

Yes.

11) Can shader also change?

Yes.

12) Why dynamic material useful?

Customization.

13) Where used in AR?

Product preview.

14) Where used in VR?

Environment customization.

15) Why is this AR/VR practical?

Because interactive visual customization is widely used in immersive AR/VR applications.